

LAB PROGRAMS

STACK

The screenshot shows the SIMATS Online Programming Lab interface. The top header includes the SIMATS ENGINEERING logo and the text "SIMATS Online Programming Lab" with a "DS PROGRAMMING" tag. The left sidebar contains a "QUESTIONS" section with "10 / 10 completed" and a dropdown menu set to "Stack using Array". Below this is a description: "A web browser maintains a history of recently visited pages. Write a C program to implement a stack using an array and perform Push, Pop, Peek, and Display operations while handling overflow and underflow conditions. Test Case 1". The main area is the "PROGRAM EDITOR" with a C language dropdown, "Run" and "Save" buttons, and a code editor containing the following C code:

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 #define MAX 100
5
6 struct Stack {
7     int arr[MAX];
8     int top;
9     int capacity;
10 };
11
12 // Initialize stack
13 void initStack(struct Stack* stack, int size) {
14     stack->top = -1;
15     stack->capacity = size;
16 }
17
18 // Check if stack is full
19 int isFull(struct Stack* stack) {
20     return stack->top == stack->capacity - 1;
21 }
```

Below the code editor is an "OUTPUT" section showing "Stack Size = Stack Elements:". On the right, the "INPUT" section shows the test case values: "5", "10 20 30".

The screenshot shows the SIMATS Online Programming Lab interface. The top header includes the SIMATS ENGINEERING logo and the text "SIMATS Online Programming Lab" with a "DS PROGRAMMING" tag. The left sidebar contains a "QUESTIONS" section with "10 / 10 completed" and a dropdown menu set to "Stack using Linked". Below this is a description: "A document editor maintains recently performed actions to support the Undo feature. Implement a stack using a linked list and perform push, pop, and display operations. Test Case 1". The main area is the "PROGRAM EDITOR" with a C language dropdown, "Run" and "Save" buttons, and a code editor containing the following C code:

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 // Structure for a Node
5 struct Node {
6     int data;
7     struct Node* next;
8 };
9
10 struct Node* top = NULL;
11
12 // Function to push an element onto the stack
13 void push(int value) {
14     struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
15     if (!newNode) {
16         printf("Stack Overflow\n");
17         return;
18     }
19     newNode->data = value;
20     newNode->next = top;
21     top = newNode;
22 }
```

Below the code editor is an "OUTPUT" section showing the result: "300 -> 200 -> 100 -> NULL". On the right, the "INPUT" section shows the test case values: "100 200 300".



SIMATS Online Programming Lab
DS PROGRAMMING

Nithiyasri
19257208

QUESTIONS 10 / 10 completed

Stack-Postfix Expr

STACK - POSTFIX EXPRESSION

"Write a C program that uses a stack to evaluate a postfix expression and display the result. Postfix expressions are commonly used in compilers because they eliminate the need for parentheses. Input

23*5+

PROGRAM EDITOR

C

Run

Save

```
1 #include <stdio.h>
2 #include <ctype.h>
3 #include <stdlib.h>
4
5 #define MAX 100
6
7 int stack[MAX];
8 int top = -1;
9
10 void push(int val) {
11     if (top >= MAX - 1) {
12         printf("Stack Overflow\n");
13         return;
14     }
15     stack[++top] = val;
16 }
17
18 int pop() {
19     if (top < 0) {
20         printf("Stack Underflow\n");
21         return -1;
22     }
```

OUTPUT

Security Error: Disallowed code pattern detected

INPUT

23*5+



SIMATS Online Programming Lab
DS PROGRAMMING

Nithiyasri
1925720828

QUESTIONS 10 / 10 completed

Stack-Infix to Post

STACK -INFIX TO POSTFIX

"Arithmetic expressions entered by programmers are generally in infix form. Before evaluation, many systems convert them into postfix notation. Write a C program using a stack to convert an infix expression into its equivalent postfix expression. Test Case 1

PROGRAM EDITOR

C

Run

Save

```
1 #include <stdio.h>
2 #include <ctype.h>
3 #include <string.h>
4 #include <stdlib.h>
5
6 #define MAX 100
7
8 int top = -1;
9 char stack[MAX];
10
11 void push(char c) {
12     if (top == MAX - 1) return;
13     stack[++top] = c;
14 }
15
16 char pop() {
17     if (top == -1) return '\0';
18     return stack[top--];
19 }
20
21 char peek() {
```

OUTPUT

INPUT

A+B*C



SIMATS Online Programming Lab

DS PROGRAMMING

Nithiyasri S
192572082RB

QUESTIONS 10 / 10 completed

Stack - Balanced Pa

STACK-BALANCED PARENTHESES

"Compilers must verify whether all opening parentheses have corresponding closing parentheses. Write a C program using a stack to check whether an expression contains balanced parentheses. Input

(A+B)*(C-D)

Output

PROGRAM EDITOR

C

RunSave

```
1 #include <stdio.h>
2 #include <string.h>
3
4 #define MAX 100
5
6 struct Stack {
7     int top;
8     char items[MAX];
9 };
10
11 void push(struct Stack *s, char c) {
12     if (s->top < MAX - 1) {
13         s->items[++(s->top)] = c;
14     }
15 }
16
17 char pop(struct Stack *s) {
18     if (s->top != -1) {
19         return s->items[(s->top)--];
20     }
21     return '\0';
22 }
```

OUTPUT

INPUT

(A+B)*(C-D)



SIMATS Online Programming Lab

DS PROGRAMMING

Nithiyasri S
192572082RB

QUESTIONS 10 / 10 completed

Stack - Reverse Stri

STACK - REVERSE STRING

"A text-processing application requires reversing user-entered strings. Write a C program that uses a stack to reverse a given string. Input

DATASTRUCTURE

Output

ERUTCURTSATAD

PROGRAM EDITOR

C

RunSave

```
1 #include <stdio.h>
2 #include <string.h>
3 #include <stdlib.h>
4
5 #define MAX 100
6
7 // Structure for the Stack
8 struct Stack {
9     int top;
10    unsigned capacity;
11    char* array;
12 };
13
14 // Function to create a stack
15 struct Stack* createStack(unsigned capacity) {
16     struct Stack* stack = (struct Stack*)malloc(sizeof(struct Stack));
17     stack->capacity = capacity;
18     stack->top = -1;
19     stack->array = (char*)malloc(stack->capacity * sizeof(char));
20     return stack;
21 }
```

OUTPUT

ERUTCURTSATAD

INPUT

DATASTRUCTURE



SIMATS Online Programming Lab
DS PROGRAMMING

Nithiyasri S
192572082RB

QUESTIONS 10 / 10 completed

Stack- Decimal to f

STACK- DECIMAL TO BINARY

"Computer systems internally represent numbers in binary format. Write a C program that uses a stack to convert a decimal number into its binary equivalent.
Input
13
Output

PROGRAM EDITOR

C

Run

Save

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 #define MAX 32
5
6 // Structure for the stack
7 struct Stack {
8     int top;
9     int items[MAX];
10 };
11
12 // Function to initialize the stack
13 void initStack(struct Stack* s) {
14     s->top = -1;
15 }
16
17 // Function to check if stack is full
18 int isFull(struct Stack* s) {
19     return s->top == MAX - 1;
20 }
21
```

OUTPUT

1101

INPUT

13



SIMATS Online Programming Lab
DS PROGRAMMING

Nithiyasri S
192572082RB

QUESTIONS 10 / 10 completed

Tower of Hanoi Prc

TOWER OF HANOI PROBLEM

"The Tower of Hanoi is a classic problem demonstrating recursion and stack behavior. Write a C program to solve the Tower of Hanoi problem and display the sequence of moves required to transfer all disks.
Input
Number of Disks = 3

PROGRAM EDITOR

C

Run

Save

```
1 #include <stdio.h>
2
3 void towerOfHanoi(int n, char from_rod, char to_rod, char aux_rod) {
4
5     if (n == 1) {
6         printf("Move Disk 1 from %c to %c\n", from_rod, to_rod);
7         return;
8     }
9
10    towerOfHanoi(n - 1, from_rod, aux_rod, to_rod);
11
12    printf("Move Disk %d from %c to %c\n", n, from_rod, to_rod);
13
14    towerOfHanoi(n - 1, aux_rod, to_rod, from_rod);
15 }
16
17 int main() {
18     int n;
19
20     printf("Number of Disks = ");
21     if (scanf("%d", &n) != 1) {
```

OUTPUT

Number of Disks = Move Disk 1 from A to C

INPUT

3

← → ↻

Not secure

simatslab.saveetha.com/practice_app/onlineide/onlineide?topic=5

SIMATS
ENGINEERING

SIMATS Online Programming Lab

DS PROGRAMMING

Nithya
1925720

QUESTIONS 10 / 10 completed

Stack - Browser His

STACK - BROWSER HISTORY

"A web browser stores visited web pages in a stack to enable navigation to previously visited pages. Write a C program that simulates browser history using a stack and supports back navigation. Input

PROGRAM EDITOR

C

Run Save

```
1 #include <stdio.h>
2 #include <string.h>
3
4 typedef struct {
5     char pages[MAX][50];
6     int top;
7 } Stack;
8
9 void push(Stack *s, char *page) {
10     if (s->top < MAX - 1) {
11         strcpy(s->pages[++(s->top)], page);
12     }
13 }
14
15 void pop(Stack *s) {
16     if (s->top >= 0) {
17         s->top--;
18     }
19 }
20
21 void displayCurrent(Stack *s) {
```

INPUT

GOOGLE
YOUTUBE
WIKIPEDIA

← → ↻

Not secure

simatslab.saveetha.com/practice_app/onlineide/onlineide?topic=5

SIMATS
ENGINEERING

SIMATS Online Programming Lab

DS PROGRAMMING

Nithya
1925720

QUESTIONS 10 / 10 completed

Stack- Undo Opera

STACK- UNDO OPERATION

"A text editor provides an Undo feature that allows users to revert the most recent action. Write a C program using a stack to simulate an Undo operation for text editing commands Input

PROGRAM EDITOR

C

Run Save

```
1 #include <stdio.h>
2 #include <string.h>
3
4 #define MAX 100
5 #define MAXSTATE 100
6
7 int main() {
8     char cmd[20];
9     char ch;
10
11     char states[MAXSTATE][MAX]; // stores text after each operation
12     int top = 0; // index of current state
13
14     // initial empty text
15     strcpy(states[0], "");
16     printf("Enter commands (Insert X or Undo). Ctrl+D/EOF to end.\n");
17
18     while (1) {
19         if (scanf("%s", cmd) != 1) break;
20
21         if (strcmp(cmd, "Insert") == 0) {
```

OUTPUT

insert a
insert b
insert c
undo